Diseño y desarrollo de servicios web - proyecto

GA7-220501096-AA5-EV03

Proyecto: Plataforma de streaming

Aprendiz: Carlos Ramirez

Programa: Análisis y Desarrollo de Software - ADSO

Ficha: 3186667

SENA

1. Descripción del proyecto

La solución desarrollada corresponde a una API REST para una plataforma de streaming. Los servicios permiten registrar e iniciar sesión, consultar y administrar el catálogo de películas y series, gestionar favoritos y guardar el progreso de reproducción de cada usuario.

2. Objetivo

Diseñar y codificar servicios web reutilizables que permitan al front-end de la plataforma de streaming consumir las funciones principales del sistema mediante solicitudes HTTP y respuestas JSON.

3. Tecnología utilizada

Elemento	Implementación
Lenguaje	PHP 8
Tipo de servicio	API REST
Formato de intercambio	JSON
Persistencia	Archivos JSON locales
Autenticación	Token Bearer generado al iniciar sesión
Versionamiento	Git

4. Ejecución del proyecto

Abrir una terminal en la carpeta del proyecto y ejecutar:

```
php -S localhost:8000 router.php
```

La API queda disponible en http://localhost:8000/api. Los servicios protegidos reciben el token de inicio de sesión en el encabezado Authorization con el formato Bearer TOKEN.

5. Documentación de servicios web

5.1. Servicio de estado

Método: GET
Endpoint: /api
Descripción: Verifica que la API esté activa.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: {"nombre":"Streaming API","estado":"activo"}

5.2. Registro de usuario

Método: POST
Endpoint: /api/auth/registro
Descripción: Crea una cuenta de usuario.

Cuerpo de solicitud

```
{"nombre":"Carlos Ramirez","correo":"carlos@email.com","contrasena":"123456"}
```

Respuesta esperada: Devuelve mensaje de registro y datos del usuario.

5.3. Inicio de sesión

Método: POST
Endpoint: /api/auth/login
Descripción: Valida credenciales y genera un token de acceso.

Cuerpo de solicitud

```
{"correo":"carlos@email.com","contrasena":"123456"}
```

Respuesta esperada: Devuelve autenticación satisfactoria, token y usuario. Si falla, responde 401.

5.4. Listado de contenidos

Método: GET
Endpoint: /api/contenidos
Descripción: Consulta el catálogo. Permite filtros buscar, tipo y genero.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve la colección de películas y series.

5.5. Detalle de contenido

Método: GET
Endpoint: /api/contenidos/{id}
Descripción: Consulta una película o serie por identificador.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve el contenido solicitado o 404.

5.6. Crear contenido

Método: POST
Endpoint: /api/contenidos
Descripción: Registra una película o serie nueva. Requiere token.

Cuerpo de solicitud

```
{"titulo":"Nueva historia","tipo":"pelicula","genero":"Drama","anio":2026,"duracion_minutos":110,"descripcion":"Descripción del contenido"}
```

Respuesta esperada: Devuelve el contenido creado con estado 201.

5.7. Actualizar contenido

Método: PUT
Endpoint: /api/contenidos/{id}
Descripción: Modifica un contenido existente. Requiere token.

Cuerpo de solicitud

```
{"titulo":"Título actualizado","genero":"Suspenso"}
```

Respuesta esperada: Devuelve el contenido actualizado o 404.

5.8. Eliminar contenido

Método: DELETE
Endpoint: /api/contenidos/{id}
Descripción: Elimina un contenido del catálogo. Requiere token.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve confirmación o 404.

5.9. Listar favoritos

Método: GET
Endpoint: /api/favoritos
Descripción: Devuelve los contenidos favoritos del usuario autenticado.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve la lista de favoritos.

5.10. Agregar favorito

Método: POST
Endpoint: /api/favoritos
Descripción: Agrega un contenido a favoritos. Requiere token.

Cuerpo de solicitud

```
{"contenido_id":1}
```

Respuesta esperada: Devuelve el favorito creado con estado 201.

5.11. Eliminar favorito

Método: DELETE
Endpoint: /api/favoritos/{id}
Descripción: Retira un registro de favoritos. Requiere token.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve confirmación o 404.

5.12. Consultar progreso

Método: GET
Endpoint: /api/progreso
Descripción: Consulta el progreso de reproducción del usuario autenticado.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve los registros de progreso.

5.13. Guardar progreso

Método: POST
Endpoint: /api/progreso
Descripción: Guarda o actualiza el segundo de reproducción de un contenido. Requiere token.

Cuerpo de solicitud

```
{"contenido_id":1,"segundo":850}
```

Respuesta esperada: Devuelve el progreso guardado o actualizado.

5.14. Eliminar progreso

Método: DELETE
Endpoint: /api/progreso/{id}
Descripción: Elimina un registro de progreso. Requiere token.

Cuerpo de solicitud

```
No requiere cuerpo.
```

Respuesta esperada: Devuelve confirmación o 404.

6. Estructura del proyecto

La aplicación se organiza separando controladores, autenticación, persistencia, punto de entrada HTTP y datos. Esto permite reutilizar los servicios y mantener cada responsabilidad en un archivo independiente.

```
CARLOS_RAMIREZ_AA5_EV03/
app/
  Controllers/
  Auth.php
  DataStore.php
  Response.php
data/
public/
  index.php
router.php
ENDPOINTS.txt
documentacion_servicios_web.pdf
enlace_repositorio.txt
```

7. Versionamiento

El proyecto debe almacenarse en un repositorio Git. Antes de realizar la entrega se debe reemplazar el contenido de enlace_repositorio.txt con la URL real del repositorio remoto utilizado para la evidencia.

8. Resultado

La API construida cubre las necesidades principales de una plataforma de streaming: autenticación de usuarios, catálogo de contenidos, favoritos y continuidad de reproducción. Los servicios responden en JSON y pueden ser consumidos por un front-end web como React.